

Filtered Approximate Nearest Neighbor Search on Vectors with Diverse Labels

1 PSEUDO CODE OF ApproximateELS

The approximated ELS retrieval method (ApproximateELS, Algorithm 1) initially checks whether $L_q \in F$ (Lines 2-6). If $L_q \in F$, it returns $\hat{F}_q = \{L_q\}$ (Line 8). Otherwise, the algorithm enumerates the nodes in $\mathbb{I}(l_{\max})$, where $l_{\max}$ is the queried label with the largest ID (Lines 9-10). Each such node is treated as a possible anchor on a trie path that may contain all labels in L_q . The function BOTTOMUP-TRAVERSAL then walks from the anchor toward the root and matches query labels in reverse ID order (Lines 12-21). During this traversal, ancestor labels with IDs larger than the current target label are skipped, a matched label advances the query pointer, and the traversal is pruned once a smaller label or the root is reached before all query labels are matched (Lines 16-21). If all query labels are matched, the anchor is regarded as a valid starting point for relaxed ELSs (Lines 22-23).

Finally, COLLECTCANDIDATEELSs performs a breadth-first traversal from the validated anchor node and inserts every encountered terminal label set into $\hat{F}_q^{(a)}$ (Lines 24-32). Note that, since each label set in F corresponds to a unique terminal node in the trie and $\hat{F}_q^{(a)}$ is maintained as a set, repeated collections of the same label set, if any, are automatically removed by the insert operation. Unlike exact ELS retrieval, ApproximateELS does not further filter these candidates to retain only minimal supersets of L_q . Therefore, the returned set may contain non-minimal supersets, but each collected terminal label set remains compatible with the query-label containment condition.

Time complexity of ApproximateELS. Lines 1-8 check whether L_q itself is a terminal label set in the trie tree, which takes $O(|L_q|)$ time. Then, Lines 9-10 enumerate all nodes in $\mathbb{I}(l_{\max})$, and Lines 13-21 perform a bottom-up traversal from each enumerated node. Since each trie path has length at most $|\mathcal{L}|$, this part costs

$$O(|\mathcal{L}| \cdot |\mathbb{I}(l_{\max})|). \quad (1)$$

For every validated anchor node, Lines 24-32 collect candidate ELSs by downward traversals. Let Υ denote the number of such downward traversal paths. Since each path costs at most $O(|\mathcal{L}|)$, this part costs

$$O(|\mathcal{L}| \cdot \Upsilon). \quad (2)$$

Therefore, the total time complexity of ApproximateELS is

$$O(|\mathcal{L}| \cdot (|\mathbb{I}(l_{\max})| + \Upsilon)), \quad (3)$$

where the $O(|L_q|)$ checking cost is dominated by the trie traversal cost. Compared with the exact ELS retrieval method, ApproximateELS skips the minimal-superset filtering step, and therefore has a smaller time complexity.

2 THEOREM AND PROOF

THEOREM 1. *For every label set $L \in \hat{F}_q^{(a)}$ such that $\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{\text{pass}}$, by employing at least one entry vector whose associated label set is L , the traversal on the unified graph can reach all qualified vectors without encountering any unqualified ones.*

PROOF. First, for any $S \subseteq F$, we use $\text{Desc}^+(S)$ to represent the group of label sets that can be reached by a traversal of LNG starting

Algorithm 1 ApproximateELS

Input: trie tree T with root r , inverted list $\mathbb{I}$, and the queried label set L_q

Output: the set $\hat{F}_q^{(a)}$ of approximated ELSs

```

1:  $\hat{F}_q^{(a)} \leftarrow \emptyset$ , current node  $c \leftarrow \text{root } r$  of  $T$ 
2: for each queried label  $l \in L_q$  in the increasing order of IDs do
3:   if no child of  $c$  corresponds to  $l$  then
4:     break
5:   else
6:      $c \leftarrow$  child of  $c$  that corresponds to  $l$ 
7: if  $c$  is a terminal node that corresponds to  $L_q$  then
8:   return  $\hat{F}_q^{(a)} = \{L_q\}$ 
9: for each node  $u \in \mathbb{I}(l_{\max})$  do
10:  BOTTOMUP-TRAVERSAL( $L_q, u$ )
11: return  $\hat{F}_q^{(a)}$ 
12: function BOTTOMUP-TRAVERSAL( $L_q, u$ )
13:   $u_{\text{initial}} \leftarrow u, i \leftarrow |L_q| - 1$ 
14:  while  $i \geq 0$  do
15:     $u_p \leftarrow$  parent of  $u$  on the trie tree
16:    if  $u_p$  matches a label with a larger ID than  $L_q[i]$  then
17:       $u \leftarrow u_p$ 
18:    else if  $u_p$  matches  $L_q[i]$  then
19:       $u \leftarrow u_p, i \leftarrow i - 1$ 
20:    else if  $u_p$  matches a label with a smaller ID than  $L_q[i]$  or  $u_p$  is  $r$  then
21:      break
22:    if  $i < 0$  then
23:      COLLECTCANDIDATEELSs( $u_{\text{initial}}$ )
24: function COLLECTCANDIDATEELSs( $u_i$ )
25:   $Q \leftarrow \{u_i\}$ 
26:  while  $Q \neq \emptyset$  do
27:     $v \leftarrow Q.\text{dequeue}()$ 
28:    if  $v$  is a terminal node corresponding to  $L \in F$  then
29:       $\hat{F}_q^{(a)}.insert(L)$ 
30:    else
31:      for each child  $x$  of  $v$  on  $T$  do
32:         $Q.enqueue(x)$ 

```

from nodes that correspond to label sets inside S . Since $\hat{F}_q$ is the set of minimum supersets of L_q on LNG, we have

$$\text{Desc}^+(\hat{F}_q) = F_{\text{pass}}. \quad (4)$$

Moreover, since $\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{\text{pass}}$, we have

$$\text{Desc}^+(\hat{F}_q) \subseteq \text{Desc}^+(\hat{F}_q^{(a)}) \subseteq \text{Desc}^+(F_{\text{pass}}). \quad (5)$$

Furthermore, given that edges on LNG indicate label set containment relations, we have

$$\text{Desc}^+(F_{\text{pass}}) = F_{\text{pass}}. \quad (6)$$

The above equations indicate that

$$\text{Desc}^+(\hat{F}_q^{(a)}) = F_{\text{pass}}. \quad (7)$$

That is to say, a traversal of LNG starting from nodes that correspond to label sets inside $\hat{F}_q^{(a)}$ reaches the exact set of qualified label sets. Further consider a traversal of the unified graph starting from at least one entry vector whose associated label set is L . The combination of cross-group and intra-group proximity edges enables this traversal to exactly reach all vectors whose associated label set is inside $\text{Desc}^+(\{L\})$. As a result, Equation (7) indicates that, the traversal on the unified graph can reach all qualified vectors without encountering any unqualified ones, by employing at least

one entry vector whose associated label set is L , for every $L \in \hat{F}_q^{(a)}$. Hence, this theorem holds. $\square$

THEOREM 2. *ApproximateELS returns a set $\hat{F}_q^{(a)}$ of qualified label sets such that*

$$\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}, \quad (8)$$

where $F_{pass} = \{L \in F \mid L_q \subseteq L\}$.

PROOF. We prove the theorem by showing the two containment relations.

First, we show that $\hat{F}_q \subseteq \hat{F}_q^{(a)}$. If $L_q \in F$, then L_q itself is the only minimum superset of L_q in F . In this case, both the exact ELS retrieval method and ApproximateELS return $\{L_q\}$, and thus the claim holds. Otherwise, when $L_q \notin F$, the exact ELS retrieval method first enumerates candidate label sets whose corresponding trie paths contain all labels in L_q , and then removes non-minimal supersets to obtain the exact ELS set $\hat{F}_q$. In contrast, ApproximateELS keeps these candidate label sets without applying the minimality filtering step. Therefore, every exact ELS is retained by ApproximateELS, which gives

$$\hat{F}_q \subseteq \hat{F}_q^{(a)}. \quad (9)$$

Second, we show that $\hat{F}_q^{(a)} \subseteq F_{pass}$. For any $L \in \hat{F}_q^{(a)}$, ApproximateELS inserts L only after the corresponding trie path has been verified to contain all labels in L_q . Since each terminal node in the

trie corresponds to a unique label set in F , this implies

$$L_q \subseteq L. \quad (10)$$

By the definition of F_{pass} , we have $L \in F_{pass}$. Hence,

$$\hat{F}_q^{(a)} \subseteq F_{pass}. \quad (11)$$

Combining Equations 9 and 11, we obtain

$$\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}. \quad (12)$$

Thus, the theorem holds. $\square$

3 XGBOOST ROUTER TRAINING

We use XGBoost, a strong classifier composed of multiple decision trees, as the router of SODA and SODA+. We set the maximum depth of each tree to 6, the learning rate to 0.05, and the number of boosting rounds to 300. We generate 40,000 queries for training, including 5,000 queries from each dataset. For each query, we select the algorithm with the highest QPS among those whose recall is above 0.9, and use its algorithm label as the training target. This labeling process is included in the training time and is very fast.

Specifically, for each dataset, we use 80% of its queries together with the queries from the other seven datasets as the training set, and use the remaining 20% as the test set to evaluate the routing performance of XGBoost. Note that these queries are only used for router training and are not included in the final experimental evaluation.